

Question 1

(i) Pearson Correlation Analysis

to analysis the relation between **general impression score** and **criteria percentage score**, the **Pearson correlation coefficient** is first calculated to measure the linear relationship between the two variables.

The Pearson correlation coefficient is defined as:

$$r = \frac{E[(X - \mu_X)(Y - \mu_Y)]}{\sqrt{E[(X - \mu_X)^2]E[(Y - \mu_Y)^2]}} \tag{1}$$

where:

- $E[\cdot]$ is Expectation
- μ_X, μ_Y represent the mean values of the variables.

The coefficient (r) ranges within **[-1, 1]**. When r approaches **1**, it indicates a strong positive correlation between the two variables; when r approaches **-1**, it indicates a strong negative correlation. When r is close to **0**, it suggests that there is little or no linear relationship between the two variables.

The calculated results are presented in the table below.

Table 1: Pearson correlation coefficients for each TOE view

View	1	2	3	4	5	6	7	8	9	10
Pearson r	0.927	0.787	0.817	0.655	0.870	0.799	0.796	0.709	0.927	0.870

From the results, the results indicate that there is a relatively strong positive correlation between **general impression** and **criteria percentage**. This indicates that when experts rate the overall image quality higher, the image typically satisfies a greater number of objective criteria.

Specifically:

- **View 1 and View 9 show the highest correlation ($r \approx 0.93$)**
- **View 4 shows the lowest correlation ($r \approx 0.66$)**

Therefore, it can be concluded that the two scoring systems demonstrate high consistency in **View 1 and View 9**.

(ii) Linear Regression Analysis

To further investigate the relationship between the two scoring methods, a **linear regression model** is constructed for each view. In this model:

- the general impression score is used as the independent variable
- the criteria percentage score is used as the dependent variable

The linear regression model is defined as:

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i \tag{2}$$

Where:

- x_i : general impression score
- y_i : criteria percentage score
- ϵ_i : random error term
- β_0 : intercept
- β_1 : regression coefficient.

To evaluate the performance of the regression model, two metrics are used: **RMSE (Root Mean Square Error)** and **R² (coefficient of determination)**.

These metrics are defined as:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}, \quad R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

(3)

- **RMSE** measures the magnitude of prediction errors
- **R²** indicates the proportion of variance in the data explained by the model.

The regression analysis results are summarised below.

Table 2: Regression performance metrics

Metric	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10
RMSE	5.87	9.10	9.61	14.34	9.31	17.11	17.77	10.69	12.82	12.06
R ²	0.860	0.619	0.667	0.428	0.757	0.639	0.634	0.503	0.859	0.757

The results show that:

- View 1 and View 9 achieve the best regression performance ($R^2 \approx 0.86$)
- View 4 shows the weakest performance ($R^2 \approx 0.43$).

Overall, most views achieve **R² values above 0.6**, indicating that the **general impression score** has a relatively strong predictive capability for the **criteria percentage score**. This further confirms the strong consistency between the two scoring approaches.

(iii) True vs Estimated Analysis

Based on the regression results, the **three views with the highest R² values** are selected for further analysis: **View 1, View 9, and View 10**. For these views, the relationship between the **true values** and the **estimated values** is visualised.



Figure 1: True vs Estimated Scores for the Three Best Performing Views with highest R^2 values for View 1-10.

From the figure, it can be observed that most data points lie close to the **ideal line** $y = x$, indicating that the regression model can effectively predict the **criteria percentage score**.

Further examination of different score ranges shows that:

In the **40–70 score range**, the data points are relatively scattered, suggesting larger prediction errors for medium-quality images. This may be because image quality differences within this range are more complex, leading to inconsistencies between subjective evaluations and objective criteria.

In the **70–100 score range**, the data points are closer to the ideal line, indicating a more stable linear relationship between the two scoring methods for high-quality images. Therefore, the linear regression model provides more accurate predictions of the **criteria percentage score** when the image quality is high.

Question 2

(i) Image similarity analysis

To evaluate the content similarity between the test images and the gold standard images, three commonly used image similarity metrics were used in this report: **Structural Similarity Index (SSI)**, **Mutual Information (MI)**, and **Cosine Similarity (CS)**.

SSI measures image quality by comparing two images in terms of **luminance, contrast, and structural similarity**. MI is an **entropy-based metric** that quantifies the amount of shared information between two images. CS evaluates similarity by computing the **cosine of the angle between the vectorized representations of two images**, indicating the consistency of their directions.

For each test image in the dataset, the SSI, MI, and CS values were computed with respect to the corresponding gold standard image for the same view. Subsequently, within each view, the test images were ranked according to their similarity scores for each metric, and the **three most similar test images** were selected. **Table 3** lists the top-3 participants whose images are most similar to the gold standard image for each view.

Table 3: Top-3 most similar test images for each view

View	SSI Top-3	MI Top-3	CS Top-3
1	P2, P4, P12	P2, P4, P6	P2, P4, P6
2	P14, P19, P8	P3, P14, P19	P14, P19, P6
3	P14, P9, P20	P3, P9, P14	P9, P14, P17
4	P2, P15, P14	P2, P5, P3	P2, P5, P3
5	P2, P16, P3	P2, P3, P5	P2, P3, P5
6	P2, P19, P1	P2, P19, P12	P2, P19, P12
7	P17, P8, P20	P17, P10, P2	P17, P10, P2
8	P2, P4, P6	P4, P2, P18	P4, P2, P18
9	P8, P9, P17	P19, P3, P14	P19, P3, P14
10	P5, P2, P11	P5, P2, P10	P5, P2, P10

From Table 3, it can be observed that some participants repeatedly appear in the Top-3 results across multiple views. For instance, **participant P2** achieves high similarity scores in several views, indicating that the ultrasound images captured by this participant are structurally and visually closer to the gold standard images, suggesting **better probe positioning and acquisition quality**.

Furthermore, the ranking results obtained by different similarity metrics show a certain level of consistency in many cases. For example, in **View 1, View 5, and View 6**, the Top-3 results produced by MI and CS are identical. By contrast, **SSI** sometimes yields different participant orders, which may be due to the fact that different similarity metrics capture different aspects of image resemblance.

(ii) Expert vs Novice statistical analysis

To evaluate the difference in image similarity between the **expert group** and the **novice group**, the following statistical hypotheses were defined for each view:

- **Null hypothesis (H_0):** There is **no significant difference** between the expert and novice groups in the mean value of a given similarity metric (SSI, MI, or CS).
- **Alternative hypothesis (H_1):** There **exists a significant difference** between the expert and novice groups in the mean value of the given metric.

An **independent sample t-test** was applied to compare the two groups, and the corresponding **p-values** were calculated. A result was considered statistically significant when **$p < 0.05$** . **Table 4** presents the p-values obtained for each view and each similarity metric.

Table 4: p-values from independent t-tests comparing expert and novice groups

View	SSI p-value	MI p-value	CS p-value
1	0.132	0.001	0.001
2	0.622	0.818	0.542
3	0.056	0.444	0.862
4	0.884	0.024	0.034
5	0.226	0.111	0.099
6	0.566	0.179	0.158
7	0.159	0.734	0.782
8	0.007	0.013	0.022
9	0.317	0.596	0.551
10	0.354	0.228	0.324

Table 4 shows that in most views, the differences between the expert and the novice are not statistically significant. However, significant differences can be observed in some views. For example, in **View 8**, the p-values of **SSI**, **MI**, and **CS** are all smaller than 0.05, indicating a statistically significant difference between experts and novices in this view. In addition, in **View 1 and View 4**, the **MI** and **CS** metrics also detect significant differences.

Overall, **Mutual Information (MI)** and **Cosine Similarity (CS)** are able to identify significant differences between experts and novices in several views, whereas **SSI shows no significant difference in most views**. Therefore, in this dataset, **MI and CS demonstrate stronger capability in distinguishing operator experience levels**, while **SSI appears to have relatively lower sensitivity for this task**.

Question 3

Q3(i) Correlation between similarity metrics (per view)

To analyse the consistency between the three image similarity metrics, the **Pearson correlation coefficients** between **SSI**, **MI**, and **CS** were calculated for each view. The results are shown below.

Table 5: Pearson correlation coefficients between SSIM, MI and CS for each view.

Metric	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10
SSIM-MI	0.753	0.457	0.318	0.489	0.716	0.762	0.494	0.737	0.014	0.825
SSIM-C	0.728	0.758	0.512	0.521	0.772	0.775	0.494	0.720	-0.070	0.854
MI-CS	0.993	0.781	0.726	0.996	0.980	0.986	0.991	0.989	0.986	0.986

The results indicate that:

- **MI and CS exhibit extremely high consistency across all views** (approximately 0.98–0.996), suggesting that these two metrics behave very similarly when measuring image content similarity.

- **The correlation between SSI and the other metrics varies significantly**, with noticeably lower correlations in some views (e.g., View 3 and View 9).
- **View 10 shows the highest overall consistency**, where both SSI-MI and SSI-CS reach relatively high correlation values.
- **View 9 shows the lowest consistency**, where SSI is almost uncorrelated with the other metrics, indicating that SSI is more sensitive to structural differences in this view.

Overall, **MI and CS demonstrate the strongest consistency, whereas the relationship between SSI and the other metrics is more dependent on the specific view**. This suggests that different similarity metrics have varying sensitivities when describing structural characteristics in images.

Q3(ii) Regularized Polynomial Regression

To predict the manual evaluation scores **general impression** and **criteria percentage**, polynomial regression models were constructed using the similarity metrics **SSI**, **MI**, and **CS** as predictors. For each view, a **7th-degree polynomial regression model** was used to capture potential non-linear relationships between the similarity metric x and the manual score y :

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_7 x^7 \quad (4)$$

Overfitting Identification and Regularization

Because a **7th-degree polynomial** contains many parameters relative to the small dataset size, the regression curve may fit the training samples too closely and produce oscillatory behaviour between data points. Such behaviour indicates **overfitting**, where the model captures noise rather than the underlying relationship.

Overfitting was identified by examining the **shape of the regression curves** and the instability of high-order polynomial coefficients. When a model produces unrealistic oscillations or extreme curvature between neighbouring samples, it indicates excessive model complexity.

To mitigate this issue, **Ridge regularization** was applied. Ridge regression penalizes large coefficient magnitudes and therefore constrains the flexibility of the polynomial model. This produces **smoother regression curves** while still allowing the model to capture non-linear relationships between similarity metrics and subjective scores.

Ridge regression introduces an **L2 penalty term** to the least-squares objective function:

$$L(\beta) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p \beta_j^2 \quad (5)$$

where:

- λ is the regularization parameter
- Larger λ values impose stronger constraints on the regression coefficients.

A total of **60 regression models** were evaluated, corresponding to:

$$10 \text{ views} \times 3 \text{ similarity metrics} \times 2 \text{ score types} \quad (6)$$

All models used a **7th-degree polynomial with Ridge regularization**. The regression models were ranked according to R^2 (**descending**) and **RMSE (ascending)**. The **three best-performing models** were then selected for visualization.

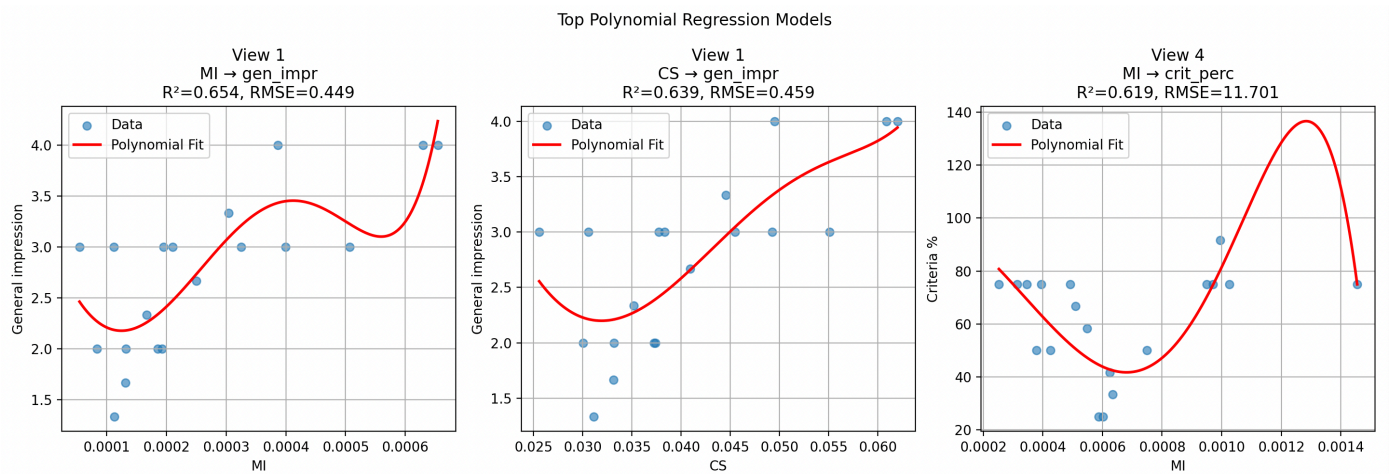


Figure 2: Polynomial regression results for the three best-performing cases.

**Table 6: Best performing regularized polynomial regression results

Best Performing Polynomial Regression Models

Metric → Score	View	Degree	R^2	RMSE
MI → General Impression	View 1	7	0.654	0.449
CS → General Impression	View 1	7	0.639	0.459
MI → Criteria %	View 4	7	0.619	11.701

- Both **Mutual Information (MI)** and **Cosine Similarity (CS)** show strong predictive ability for **General Impression** in **View 1**, achieving R^2 values of **0.654** and **0.639**, respectively. This indicates that higher similarity between the reconstructed and reference images is associated with higher subjective visual quality scores.
- The fitted regression curves exhibit clear non-linear patterns, suggesting that the relationship between similarity metrics and perceptual scores cannot be adequately captured by linear models. The 7th-degree polynomial enables the model to represent these curved trends in the data.
- Without regularization, high-degree polynomials can produce oscillatory behavior between data points. The use of **Ridge regularization** stabilizes the regression curves and prevents extreme coefficient values, leading to smoother and more reliable predictions.
- The best regression models are concentrated in **View 1** and **View 4**, suggesting that the relationship between similarity metrics and subjective evaluation varies across different views of the object.

Q3(iii) Gaussian Basis Regression

Gaussian basis regression uses multiple Gaussian functions to more flexibly model **non-linear relationships**. By adjusting the **number of Gaussian basis functions M** and the **Ridge regularization parameter α** , a balance between overfitting and underfitting can be achieved.

Gaussian basis regression are defined as

$$\phi_j(x) = \exp\left(-\frac{(x - \mu_j)^2}{2\sigma^2}\right) \quad (7)$$

where:

- μ_j is the centre of the j -th Gaussian basis
- σ represents the width of the basis function
- M is the number of Gaussian basis functions.

By tuning **the number of basis functions M** and the **Ridge regularization parameter α** , the trade-off between model complexity and generalization ability can be controlled.

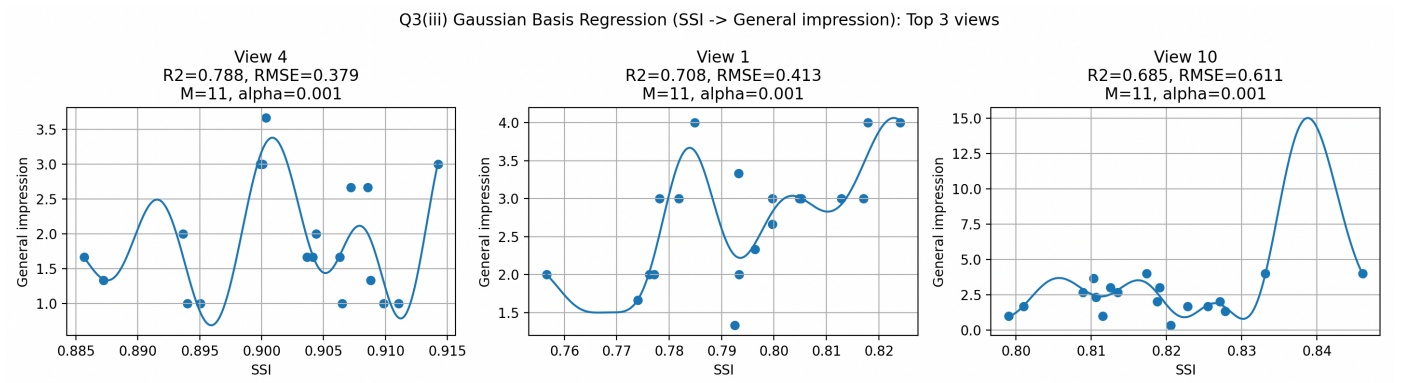


Figure 3: Gaussian basis regression results for the best-performing views.

The three best-performing views are shown below.

Table 7: best performance top 3

View	M	Alpha	R ²	RMSE
View 4	11	0.001	0.788	0.379
View 1	11	0.001	0.708	0.413
View 10	11	0.001	0.685	0.611

The result shows:

- **View 4 achieves the best model performance**, with an R^2 value of **0.788**.
- Compared with polynomial regression, **Gaussian basis regression better captures the non-linear relationship between SSI and the scores**.
- Different views require same numbers of basis functions. For example, **View 1 and 5 requires more basis functions (M=11)**.

Overall, **Gaussian basis regression performs better than polynomial regression when modelling the relationship between SSI and general impression**, indicating that localized non-linear basis functions provide stronger representational capability for complex relationships.

Question 4 Image Misalignment Analysis

As described in the introduction, high-quality TOE images should position the cardiac structures near the centre of the heart scanning area. Therefore, the degree of misalignment between a test image and the corresponding gold standard image can be used to evaluate image quality. This misalignment can be represented by a **rigid transformation** consisting of two parameters: **rotation** and **translation**.

Q4(i) Rotation and Translation Calculation

In this experiment, a **Euclidean motion model** is adopted. This model contains only rotation and translation degrees of freedom, and can therefore be represented by a **2×3 transformation matrix**:

$$M = \begin{bmatrix} M_{11} & M_{12} & M_{13} \\ M_{21} & M_{22} & M_{23} \end{bmatrix} \tag{8}$$

$$M = \begin{bmatrix} M_{11} & M_{12} & M_{13} \\ M_{21} & M_{22} & M_{23} \end{bmatrix} \tag{9}$$

where

- $M_{11}, M_{12}, M_{21}, M_{22}$ represent the **rotation components**
- M_{13}, M_{23} represent the **translation vector**

Rotation and Translation Computation

According to the formulas provided in the assignment, the rotation angle and the Euclidean distance of the translation vector are computed as:

$$\theta = \tan^{-1} \left(\frac{M_{21}}{M_{11}} \right), \quad T = \sqrt{M_{13}^2 + M_{23}^2} \tag{10}$$

The rotation angle is finally converted to **degrees**. The value T represents the overall displacement of the test image relative to the gold standard image.

Table 8: Example rotation and translation values

participant	view	rotation	translation	GI	criteria
1	1	-3.7558	29.9141	4.00	100
1	2	0.6953	6.1488	4.00	100
1	3	4.6807	15.5152	4.00	100
1	4	4.8971	16.6586	3.00	75
1	5	-7.5372	29.7003	3.67	88.89

Table 8 shows example results of the calculated **rotation** and **translation** values for several participants across different views.

Q4(ii) Statistical Test Between Experts and Novices

Participants were divided into two groups:

- **Experts:** participant 1–7
- **Novices:** participant 8–20

For each view, independent sample **t-tests** were performed on both **rotation** and **translation**.

The hypotheses are defined as:

- **H0:** There is no significant difference between experts and novices.
- **H1:** There is a significant difference between experts and novices.

In other words, the null hypothesis assumes that experts and novices do not differ significantly in terms of **rotation** or **translation**.

Table 9: Statistical test results between experts and novices

View	Rotation p-value	Translation p-value
1	0.9356	0.3189
2	0.7777	0.3178
3	0.5533	0.6660
4	0.9146	0.7418
5	0.2872	0.4568
6	0.4465	0.8081
7	0.1511	0.7944
8	0.7709	0.9573
9	0.2822	0.9042
10	0.9127	0.1444

Result Analysis

Based on the statistical results, under a significance level of $\alpha = 0.05$, all p-values are **greater than 0.05**, therefore the null hypothesis cannot be rejected.

This indicates that, in this dataset, **no significant difference in image misalignment (rotation or translation) is observed between experts and novices**.

Q4(iii) Linear Regression Analysis

Linear regression was performed using *rotation* and *translation* as independent variables against the two types of manual scores. The linear regression model is shown in Equation (·), while the formulas for *RMSE* and *R²* are given in Equation (3).

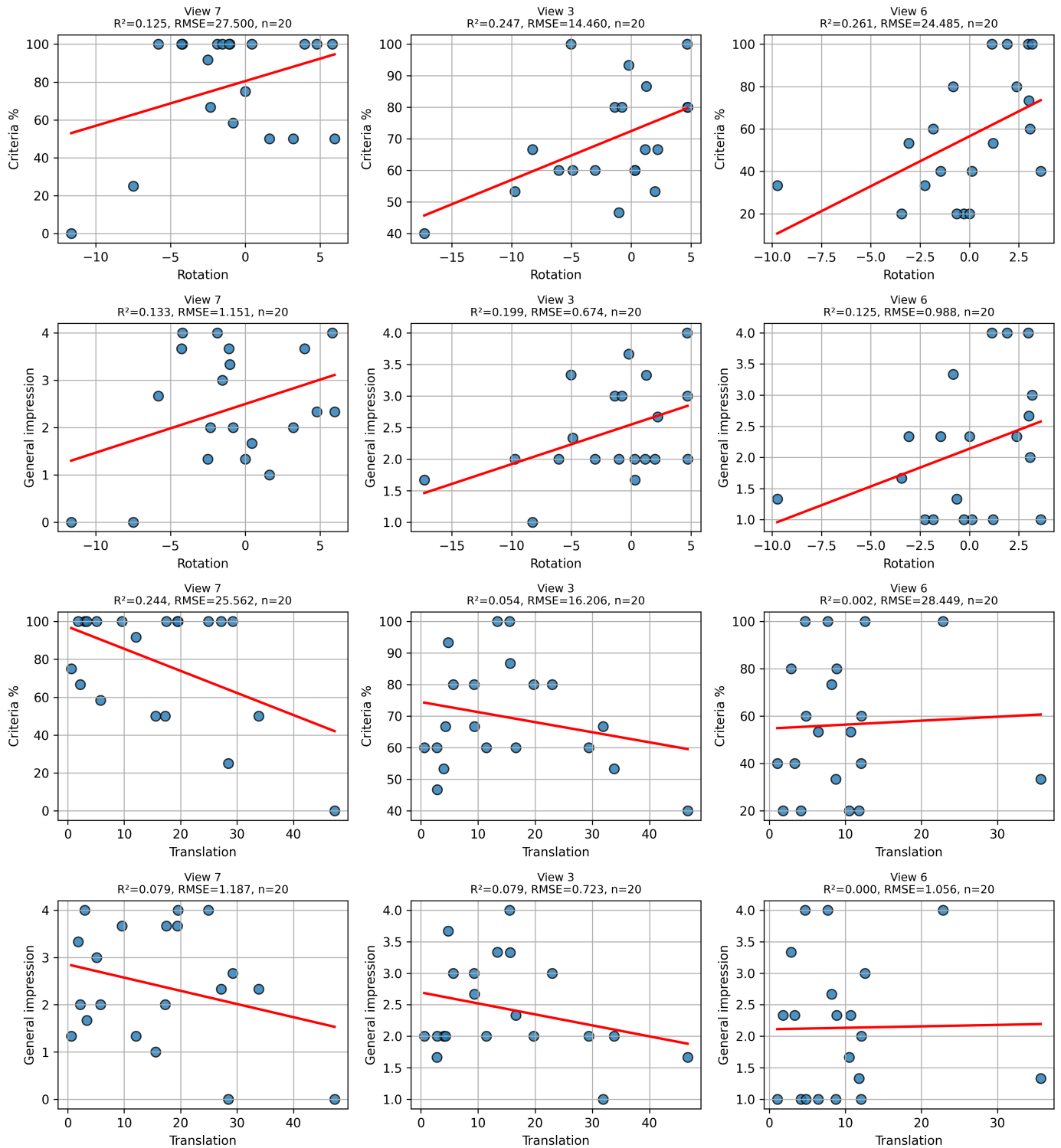
For each view, calculate **RMSE** and **R²**. Sort by the R² value. The three best-performing models are 3, 6, and 7, as shown in Table 10:

Table 10: Top 3 Regression Results

View	Independent	Dependent	RMSE	R ²
6	rotation	criteria	24.485	0.2608
3	rotation	criteria	14.460	0.2470
7	translation	criteria	25.562	0.2437

All regression models were evaluated by calculating **RMSE** and **R²**, which were used to identify the best-performing models. The models were ranked according to their **R² values**, and the three best-performing models are shown in **Table X**.

Q4(iii) Linear Regression Results



Discussion of Results

From the regression results, it can be observed that most models exhibit relatively low **R² values** (approximately **0.10–0.25**), indicating that **rotation and translation explain only a small portion of the variation in image quality scores**.

This suggests that image quality is not determined solely by probe positioning, but may also be influenced by several other factors, including:

- Variations in cardiac anatomical structure
- Image acquisition angle
- Operator handling stability
- Imaging device parameters

Therefore, using only **image misalignment metrics** is insufficient to fully predict image quality.

Appendix

Appendix provides copy of **all code used** as an **Appendix**.

Question 1

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import pearsonr
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np
import scipy.io as sio

def load_toe_dataset(mat_path: str) -> dict:
    data = sio.loadmat(mat_path)

    test_img = data["test_img"]
    gold_img = data["gold_img"][0]
    gen_impr = data["gen_impr"]
    crit_perc = data["crit_perc"]

    valid_mask = np.ones((20, 10), dtype=bool)

    for p in range(20):
        for v in range(10):

            img = test_img[p][v]

            g = gen_impr[p][v]
            c = crit_perc[p][v]

            # Mark sample as invalid if image or score is missing
            if (
                img is None
                or img.size == 0
                or g == -1
                or c == -1
            ):
                valid_mask[p, v] = False

    dataset = {
        "test_img": test_img,
```

```

        "gold_img": gold_img,
        "gen_impr": gen_impr,
        "crit_perc": crit_perc,
        "valid_mask": valid_mask
    }

    return dataset

def q1_i_correlation(dataset):

    gen_impr = dataset["gen_impr"]
    crit_perc = dataset["crit_perc"]
    valid_mask = dataset["valid_mask"]

    pearson_scores = []

    for view in range(10):

        general = []
        criteria = []

        for p in range(20):

            if not valid_mask[p, view]:
                continue

            general.append(gen_impr[p, view])
            criteria.append(crit_perc[p, view])

        general = np.array(general)
        criteria = np.array(criteria)

        r, _ = pearsonr(general, criteria)

        pearson_scores.append(r)

    return pearson_scores

def q1_ii_regression(dataset):

    gen_impr = dataset["gen_impr"]
    crit_perc = dataset["crit_perc"]
    valid_mask = dataset["valid_mask"]

    rmse_scores = []
    r2_scores = []
    predictions = {}

    for view in range(10):

        general = []
        criteria = []

```

```

for p in range(20):

    if not valid_mask[p, view]:
        continue

    general.append(gen_impr[p, view])
    criteria.append(crit_perc[p, view])

general = np.array(general)
criteria = np.array(criteria)

X = general.reshape(-1,1)
y = criteria

model = LinearRegression()
model.fit(X,y)

y_pred = model.predict(X)

predictions[view] = (y, y_pred)

rmse = np.sqrt(mean_squared_error(y,y_pred))
r2 = r2_score(y,y_pred)

rmse_scores.append(rmse)
r2_scores.append(r2)

return rmse_scores, r2_scores, predictions

def q1_iii_plot(predictions, r2_scores):

    best_views = np.argsort(r2_scores)[-3:][::-1]

    print("Best performing views:", best_views + 1)

    fig, axes = plt.subplots(1,3,figsize=(15,5))

    for i,view in enumerate(best_views):

        y_true, y_pred = predictions[view]

        axes[i].scatter(y_true, y_pred, s=60)

        axes[i].plot([0,100],[0,100], 'r--', linewidth=2)

        axes[i].set_title(f"View {view+1}")
        axes[i].set_xlabel("True criteria percentage")
        axes[i].set_ylabel("Estimated criteria percentage")
        axes[i].grid(True)

    plt.suptitle("True vs Estimated Scores for Best Performing Views")

```

```

plt.tight_layout()
plt.show()

if __name__ == "__main__":

    dataset = load_toe_dataset("cw2.mat")

    # Q1(i)
    pearson_scores = q1_i_correlation(dataset)

    print("\nQ1(i) Pearson correlation")
    for i,r in enumerate(pearson_scores):
        print(f"View {i+1}: {r:.3f}")

    # Q1(ii)
    rmse_scores, r2_scores, predictions = q1_ii_regression(dataset)

    print("\nQ1(ii) Regression performance")

    for i in range(10):
        print(f"View {i+1}: RMSE={rmse_scores[i]:.3f} R2={r2_scores[i]:.3f}")

    # Q1(iii)
    print("\nQ1(iii) Best view plots")
    q1_iii_plot(predictions, r2_scores)

```

Question 2

```

import numpy as np
import scipy.io as sio

from skimage.metrics import structural_similarity
from sklearn.metrics.pairwise import cosine_similarity
from scipy.stats import entropy, ttest_ind

def load_toe_dataset(mat_path: str) -> dict:

    data = sio.loadmat(mat_path)

    test_img = data["test_img"]
    gold_img = data["gold_img"][0] # (1,10) → (10,)
    gen_impr = data["gen_impr"]
    crit_perc = data["crit_perc"]

    # initialise mask indicating valid samples
    valid_mask = np.ones((20, 10), dtype=bool)

```

```

for p in range(20):
    for v in range(10):

        img = test_img[p][v]

        g = gen_impr[p][v]
        c = crit_perc[p][v]

        # mark sample as invalid if image or score is missing
        if (
            img is None
            or img.size == 0
            or g == -1
            or c == -1
        ):
            valid_mask[p, v] = False

dataset = {
    "test_img": test_img,
    "gold_img": gold_img,
    "gen_impr": gen_impr,
    "crit_perc": crit_perc,
    "valid_mask": valid_mask
}

return dataset

```

Mutual Information

```

def mutual_information(img1, img2, bins=64):

    # compute joint histogram of the two images
    hist_2d, _, _ = np.histogram2d(
        img1.ravel(),
        img2.ravel(),
        bins=bins
    )

    # normalise to obtain joint probability distribution
    pxy = hist_2d / np.sum(hist_2d)

    # marginal probability distributions
    px = np.sum(pxy, axis=1)
    py = np.sum(pxy, axis=0)

    # compute entropy terms
    Hx = entropy(px + 1e-10)
    Hy = entropy(py + 1e-10)
    Hxy = entropy(pxy.flatten() + 1e-10)

    return Hx + Hy - Hxy

```

```

# compute similarity metrics
def compute_similarity(cw2_data):

    test_img = cw2_data["test_img"]
    gold_img = cw2_data["gold_img"][0]

    results = []

    for participant in range(20):

        group = "expert" if participant < 7 else "novice"

        for view in range(10):

            img = test_img[participant][view]

            if img.size == 0:
                continue

            gold = gold_img[view]

            # compute similarity metrics
            img = np.squeeze(img)
            gold = np.squeeze(gold)

            # compute structural similarity (SSI)
            ssi = structural_similarity(
                img,
                gold,
                data_range=gold.max() - gold.min()
            )

            # compute structural similarity (SSI)
            mi = mutual_information(img, gold)

            # flatten images for cosine similarity
            v1 = img.flatten().reshape(1,-1)
            v2 = gold.flatten().reshape(1,-1)

            # compute cosine similarity
            cs = cosine_similarity(v1, v2)[0][0]

            results.append({
                "participant": participant,
                "view": view,
                "group": group,
                "ssi": ssi,
                "mi": mi,
                "cs": cs
            })

```

```

    return results

# top-3 similar images
def report_top3(results):

    metrics = ["ssi", "mi", "cs"]

    print("\nTop 3 most similar images for each view\n")

    for view in range(10):

        # collect results for current view
        view_data = [r for r in results if r["view"] == view]

        print(f"View {view+1}")

        for m in metrics:

            # sort participants according to similarity score
            sorted_view = sorted(
                view_data,
                key=lambda x: x[m],
                reverse=True
            )

            top3 = sorted_view[:3]

            participants = [
                f"P{r['participant']+1}"
                for r in top3
            ]

            print(f"{m.upper()} top3:", participants)

        print()

# Expert vs Novice hypothesis test
def expert_novice_test(results):

    metrics = ["ssi", "mi", "cs"]

    print("\nExpert vs Novice hypothesis test\n")

    for view in range(10):

        view_data = [r for r in results if r["view"] == view]

        print(f"View {view+1}")

        for m in metrics:

```

```

# expert vs novice hypothesis test
expert = [r[m] for r in view_data if r["group"]=="expert"]
novice = [r[m] for r in view_data if r["group"]=="novice"]

# perform two-sample t-test
stat, p = ttest_ind(expert, novice)

print(
    f"{m.upper()} | expert mean={np.mean(expert):.3f} "
    f"novice mean={np.mean(novice):.3f} "
    f"p-value={p:.4f}"
)

print()

if __name__ == "__main__":
    dataset = load_toe_dataset("cw2.mat")
    results = compute_similarity(dataset)
    report_top3(results)
    expert_novice_test(results)

```

Question 3

```

import numpy as np
import matplotlib.pyplot as plt

import scipy.io as sio
from scipy.stats import pearsonr, entropy
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error, r2_score

from skimage.metrics import structural_similarity
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

# Help function
def rmse_r2(y_true, y_pred):
    # trans to float type
    rmse = float(np.sqrt(mean_squared_error(y_true, y_pred)))
    r2 = float(r2_score(y_true, y_pred))
    return rmse, r2

def count_nonzero_coef(model, thresh=0.01):
    coef = model.coef_.copy()
    return int(np.sum(np.abs(coef) >= thresh))

def load_toe_dataset(mat_path: str) -> dict:

```

```

data = sio.loadmat(mat_path)

test_img = data["test_img"]
gold_img = data["gold_img"][0]  # (1,10) → (10,)
gen_impr = data["gen_impr"]
crit_perc = data["crit_perc"]

# initialise mask indicating valid samples
valid_mask = np.ones((20, 10), dtype=bool)

for p in range(20):
    for v in range(10):

        img = test_img[p][v]

        g = gen_impr[p][v]
        c = crit_perc[p][v]

        # mark sample as invalid if image or score is missing
        if (
            img is None
            or img.size == 0
            or g == -1
            or c == -1
        ):
            valid_mask[p, v] = False

dataset = {
    "test_img": test_img,
    "gold_img": gold_img,
    "gen_impr": gen_impr,
    "crit_perc": crit_perc,
    "valid_mask": valid_mask
}

return dataset

def mutual_information(img1, img2, bins=64):

    # Compute joint histogram of two images
    hist_2d, _, _ = np.histogram2d(img1.ravel(), img2.ravel(), bins=bins)

    # Convert histogram to joint probability distribution
    pxy = hist_2d / (np.sum(hist_2d) + 1e-12)

    # Marginal probability distributions
    px = np.sum(pxy, axis=1)
    py = np.sum(pxy, axis=0)

    # Compute entropy terms
    Hx = entropy(px + 1e-12)
    Hy = entropy(py + 1e-12)

```

```

Hxy = entropy(pxy.flatten() + 1e-12)

return Hx + Hy - Hxy

def compute_similarity(test_img, gold_img, valid_mask):

    # Initialise similarity matrices
    SSI = np.full((20, 10), np.nan, dtype=float)
    MI = np.full((20, 10), np.nan, dtype=float)
    CS = np.full((20, 10), np.nan, dtype=float)

    for p in range(20):
        for v in range(10):

            # Skip invalid samples
            if not valid_mask[p, v]:
                continue

            img = test_img[p][v]
            gold = gold_img[v]

            # Ensure valid image data
            if img is None or getattr(img, "size", 0) == 0:
                continue
            if img.shape != gold.shape:
                continue

            # Compute Structural Similarity Index
            dr = float(img.max() - img.min())
            if dr <= 0:
                dr = 1.0
            SSI[p, v] = structural_similarity(img, gold, data_range=dr)

            # Compute Mutual Information
            MI[p, v] = mutual_information(img, gold)

            # Compute Cosine Similarity between flattened images
            CS[p, v] = cosine_similarity(img.reshape(1, -1), gold.reshape(1, -1))[0, 0]

    return SSI, MI, CS

def plot_top_regressions(results, top_k=3):
    # draw top models
    top_models = results[:top_k]

    fig, axes = plt.subplots(1, top_k, figsize=(15,5))

    for i, r in enumerate(top_models):

        ax = axes[i]

        x_v = r["x"]

```

```

y_v = r["y"]
model = r["model"]

ax.scatter(x_v, y_v, alpha=0.6, label="Data")

x_grid = np.linspace(x_v.min(), x_v.max(), 400).reshape(-1,1)

y_grid = model.predict(x_grid)

ax.plot(x_grid, y_grid, color="red", linewidth=2, label="Polynomial Fit")

ylab = "Criteria %" if r["score"] == "crit_perc" else "General impression"

ax.set_xlabel(r["metric"])
ax.set_ylabel(ylab)

ax.set_title(
    f"View {r['view']+1}\n"
    f"{r['metric']} → {r['score']}\n"
    f"R²={r['r2']:.3f}, RMSE={r['rmse']:.3f}"
)

ax.grid(True)
ax.legend()

plt.suptitle("Top Polynomial Regression Models")
plt.tight_layout()
plt.show()

def gaussian_design_matrix(x, centers, sigma):

    # Construct Gaussian basis functions
    x = np.asarray(x).reshape(-1, 1)
    centers = np.asarray(centers).reshape(1, -1)

    return np.exp(-((x - centers) ** 2) / (2 * sigma ** 2))

def run_polynomial_regression_ranking(SS1, MI, CS, gen_impr, crit_perc, degree=7,
alpha=0.001):
    # calculate ii result
    metrics = [
        ("SS1", SS1),
        ("MI", MI),
        ("CS", CS)
    ]

    scores = [
        ("gen_impr", gen_impr),
        ("crit_perc", crit_perc)
    ]

    results = []

```

```

for v in range(10): # for every view
    for m_name, m_mat in metrics: # for every
        for s_name, s_mat in scores:

            x = m_mat[:, v]
            y = s_mat[:, v]

            mask = ~np.isnan(x) & (y != -1)

            x_v = x[mask].reshape(-1,1).astype(float)
            y_v = y[mask].astype(float)

            if len(x_v) < 3:
                continue

            model = make_pipeline(
                PolynomialFeatures(degree, include_bias=False),
                StandardScaler(),
                Ridge(alpha=alpha)
            )

            model.fit(x_v, y_v)

            y_hat = model.predict(x_v)

            rmse, r2 = rmse_r2(y_v, y_hat)

            results.append({
                "view": v,
                "metric": m_name,
                "score": s_name,
                "rmse": rmse,
                "r2": r2,
                "model": model,
                "x": x_v,
                "y": y_v
            })

# R2 and RMSE sort
results = sorted(
    results,
    key=lambda x: (-x["r2"], x["rmse"])
)

return results

# Load data
dataset = load_toe_dataset("cw2.mat")

test_img = dataset["test_img"]

```

```

gold_img = dataset["gold_img"]
gen_impr = dataset["gen_impr"]
crit_perc = dataset["crit_perc"]
valid_mask = dataset["valid_mask"]

# SSI, MI, CS
SSI, MI, CS = compute_similarity(test_img, gold_img, valid_mask)

# Q3(i) correlation (print only)
print("\nQ3(i) Correlation between similarity metrics (per view)\n")
for v in range(10):
    ssi = SSI[:, v]
    mi = MI[:, v]
    cs = CS[:, v]
    mask = ~np.isnan(ssi) & ~np.isnan(mi) & ~np.isnan(cs)

    r_ssi_mi, _ = pearsonr(ssi[mask], mi[mask])
    r_ssi_cs, _ = pearsonr(ssi[mask], cs[mask])
    r_mi_cs, _ = pearsonr(mi[mask], cs[mask])

    print(f"View {v+1}: SSI-MI={r_ssi_mi:.3f}, SSI-CS={r_ssi_cs:.3f}, MI-CS={r_mi_cs:.3f}")

# Q3(ii) Regularized Polynomial Regression (Modified with Scaling)
print("\nQ3(ii) Regularized Polynomial Regression\n")

results = run_polynomial_regression_ranking(
    SSI, MI, CS,
    gen_impr, crit_perc
)

print("\nTotal regressions evaluated:", len(results))
print("\nTop 10 regression models:\n")

for i, r in enumerate(results[:10]):

    print(
        f"{i+1:02d}. "
        f"View {r['view']+1} | "
        f"{r['metric']} → {r['score']} | "
        f"Polynomial degree=7 | "
        f"R²={r['r2']:.3f} | RMSE={r['rmse']:.3f}"
    )

plot_top_regressions(results, top_k=3)

# Q3(iii) Gaussian basis regression (SSI -> gen_impr)
print("\nQ3(iii) Gaussian basis regression (automatic model selection)\n")

# hyperparameter search space

```

```

M_list = range(2, 12)
alpha_list = [0.001, 0.01, 0.1, 1]

results = []

# Model selection for each view
for v in range(10):

    x = SSI[:, v]
    y = gen_impr[:, v]

    mask = ~np.isnan(x) & (y != -1)

    x_v = x[mask].astype(float)
    y_v = y[mask].astype(float)

    if len(x_v) < 3:
        continue

    best_r2 = -np.inf
    best_model = None
    best_params = None

    for M in M_list:

        centers = np.linspace(x_v.min(), x_v.max(), M)

        sigma = float((x_v.max() - x_v.min()) / M)
        if sigma <= 1e-12:
            sigma = 1.0

        Phi = gaussian_design_matrix(x_v, centers, sigma)

        for alpha in alpha_list:

            model = Ridge(alpha=alpha)
            model.fit(Phi, y_v)

            y_hat = model.predict(Phi)

            rmse, r2 = rmse_r2(y_v, y_hat)

            # keep best model for this view
            if r2 > best_r2:
                best_r2 = r2
                best_model = (centers, sigma, model)
                best_params = (M, alpha, rmse)

    results.append({
        "view": v,
        "r2": best_r2,
        "rmse": best_params[2],

```

```

        "M": best_params[0],
        "alpha": best_params[1],
        "model": best_model
    })

# Select Top 3 views
results = sorted(results, key=lambda x: x["r2"], reverse=True)

top3 = results[:3]

print("Top 3 views based on R2:")
for r in top3:
    print(
        f"View {r['view']+1} | "
        f"R2={r['r2']:.3f}, RMSE={r['rmse']:.3f}, "
        f"M={r['M']}, alpha={r['alpha']}"
    )

# Plot Top 3
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for i, r in enumerate(top3):

    v = r["view"]

    centers, sigma, model = r["model"]

    x = SSI[:, v]
    y = gen_impr[:, v]

    mask = ~np.isnan(x) & (y != -1)

    x_v = x[mask].astype(float)
    y_v = y[mask].astype(float)

    Phi = gaussian_design_matrix(x_v, centers, sigma)

    y_hat = model.predict(Phi)

    rmse, r2 = rmse_r2(y_v, y_hat)

    ax = axes[i]

    ax.scatter(x_v, y_v)

    x_grid = np.linspace(x_v.min(), x_v.max(), 300)

    Phi_g = gaussian_design_matrix(x_grid, centers, sigma)

    y_grid = model.predict(Phi_g)

```

```

ax.plot(x_grid, y_grid)

ax.set_xlabel("SSI")
ax.set_ylabel("General impression")

ax.grid(True)

ax.set_title(
    f"View {v+1}\n"
    f"R2={r2:.3f}, RMSE={rmse:.3f}\n"
    f"M={r['M']}, alpha={r['alpha']}"
)

plt.suptitle(
    "Q3(iii) Gaussian Basis Regression (SSI -> General impression): Top 3 views"
)

plt.tight_layout()

plt.show()

```

Question 4

```

import scipy.io as sio
import numpy as np
import cv2
import math
import pandas as pd
import matplotlib.pyplot as plt

from scipy.stats import ttest_ind
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# load the data
def load_toe_dataset(mat_path: str) -> dict:

    data = sio.loadmat(mat_path)

    test_img = data["test_img"]
    gold_img = data["gold_img"][0] # (1,10) -> (10,)
    gen_impr = data["gen_impr"]
    crit_perc = data["crit_perc"]

    # initialise mask indicating valid samples
    valid_mask = np.ones((20, 10), dtype=bool)

    for p in range(20):
        for v in range(10):

```

```

        img = test_img[p][v]

        g = gen_impr[p][v]
        c = crit_perc[p][v]

        # mark sample as invalid if image or score is missing
        if (
            img is None
            or img.size == 0
            or g == -1
            or c == -1
        ):
            valid_mask[p, v] = False

dataset = {
    "test_img": test_img,
    "gold_img": gold_img,
    "gen_impr": gen_impr,
    "crit_perc": crit_perc,
    "valid_mask": valid_mask
}

return dataset

# ECC, using default parameter
def compute_alignment(test, gold):
    test = test.astype(np.float32)
    gold = gold.astype(np.float32)

    sz = gold.shape

    warp_matrix = np.eye(2, 3, dtype=np.float32)

    try:

        cc, warp_matrix = cv2.findTransformECC(
            gold,
            test,
            warp_matrix,
            warp_mode,
            criteria
        )
        test_aligned = cv2.warpAffine(
            test,
            warp_matrix,
            (sz[1], sz[0]),
            flags=cv2.INTER_LINEAR + cv2.WARP_INVERSE_MAP
        )

    # translation
    tx = warp_matrix[0, 2]
    ty = warp_matrix[1, 2]

```

```

        translation = np.sqrt(tx**2 + ty**2)

        # rotation
        rotation_rad = math.atan2(
            warp_matrix[1, 0],
            warp_matrix[0, 0]
        )
        rotation_deg = np.degrees(rotation_rad)
        return rotation_deg, translation

    except cv2.error:
        return np.nan, np.nan

# Load dataset
dataset = load_toe_dataset("cw2.mat")

test_img = dataset["test_img"]
gold_img = dataset["gold_img"]
gen_impr = dataset["gen_impr"]
crit_perc = dataset["crit_perc"]
valid_mask = dataset["valid_mask"]

# parameters
warp_mode = cv2.MOTION_EUCLIDEAN

number_of_iterations = 500
termination_eps = 1e-10

criteria = (
    cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT,
    number_of_iterations,
    termination_eps,
)

# Q4(i)
records = []

for p in range(20):

    for v in range(10):

        if not valid_mask[p, v]:
            continue

        test = test_img[p][v]
        gold = gold_img[v]

        rotation, translation = compute_alignment(test, gold)

        GI = gen_impr[p][v]
        CP = crit_perc[p][v]

```

```

        records.append({
            "participant": p + 1,
            "view": v + 1,
            "rotation": rotation,
            "translation": translation,
            "GI": GI,
            "criteria": CP
        })

df = pd.DataFrame(records)

print("\nRotation & Translation values")
print(df.head())

# Remove NaN rows (important)
df = df.dropna()

# Q4(ii)
experts = df[df["participant"] <= 7]
novices = df[df["participant"] > 7]

print("\nExpert vs Novice statistical test")

for view in range(1,11):

    exp_view = experts[experts["view"] == view]
    nov_view = novices[novices["view"] == view]

    r_stat, r_p = ttest_ind(
        exp_view["rotation"],
        nov_view["rotation"],
        equal_var=False
    )

    t_stat, t_p = ttest_ind(
        exp_view["translation"],
        nov_view["translation"],
        equal_var=False
    )

    print(f"\nView {view}")
    print(f"Rotation p-value: {r_p:.4f}")
    print(f"Translation p-value: {t_p:.4f}")

# Q4(iii)
results = []

for view in range(1,11):

```

```

view_data = df[df["view"] == view]

for x_var in ["rotation", "translation"]:
    for y_var in ["criteria", "GI"]:

        data = view_data[[x_var, y_var]].dropna()

        if len(data) < 3:
            continue

        X = data[[x_var]].values
        y = data[y_var].values

        model = LinearRegression()
        model.fit(X, y)

        y_pred = model.predict(X)

        rmse = np.sqrt(mean_squared_error(y, y_pred))
        r2 = r2_score(y, y_pred)

        results.append({
            "view": view,
            "independent": x_var,
            "dependent": y_var,
            "RMSE": rmse,
            "R2": r2
        })

results_df = pd.DataFrame(results)

print("\nRegression results")
print(results_df)

# Compute average performance per view
view_score = results_df.groupby("view").agg({
    "R2": "mean",
    "RMSE": "mean"
}).reset_index()

best_views = view_score.sort_values(
    ["R2", "RMSE"],
    ascending=[False, True]
).head(3)

print("\nBest performing views")
print(best_views)

views = best_views["view"].astype(int).tolist()

combos = [

```

```

    ("rotation", "criteria"),
    ("rotation", "GI"),
    ("translation", "criteria"),
    ("translation", "GI")
]

#draw diagram
plt.figure(figsize=(12,14))

for r,(x_var,y_var) in enumerate(combos):

    for c,view in enumerate(views):

        view_data = df[df["view"] == view]
        data = view_data[[x_var,y_var]].dropna()

        if len(data) < 3:
            continue

        X = data[[x_var]].values
        y = data[y_var].values

        model = LinearRegression()
        model.fit(X,y)

        y_pred = model.predict(X)

        r2 = r2_score(y,y_pred)
        rmse = np.sqrt(mean_squared_error(y,y_pred))

        x_grid = np.linspace(X.min(),X.max(),200).reshape(-1,1)
        y_grid = model.predict(x_grid)

        index = r*3 + c + 1
        plt.subplot(4,3,index)

        plt.scatter(
            X,
            y,
            s=60,
            alpha=0.8,
            edgecolors="black"
        )

        plt.plot(
            x_grid,
            y_grid,
            color="red",
            linewidth=2
        )

        ylabel = "Criteria %" if y_var=="criteria" else "General impression"

```

```
plt.xlabel(x_var.capitalize())
plt.ylabel(ylabel)

plt.title(
    f"View {view}\nR2= {r2:.3f}, RMSE={rmse:.3f}, n={len(X)}",
    fontsize=9
)

plt.grid(True)

plt.suptitle(
    "Q4(iii) Linear Regression Results",
    fontsize=16
)

plt.tight_layout(rect=[0,0,1,0.96])
plt.show()
```